

Unit 3 — Algorithms for Data Analysis & Storing the Data

Syllabus: accepting constraints and relaxing · thinking about time · summarization techniques · long-term storage · keeping it in memory

Sources: Psaltis *Streaming Data* Ch.5 (printed pp.77–94) and Ch.6 (pp.95–112) + all question papers. **BOOK p.xx** ↗ opens *Streaming-Data[1].pdf* at printed page xx (PDF = printed + 20). Blue **highlight** = must-remember. **IMPORTANT** = asked before.

0 · Hot-list for Unit 3 (Q5/Q6 — choose one, 16 marks)

Rank	Topic	Asked in
1	Windows : sliding / tumbling, choose & justify, compare	SEE Feb'26 5a (10M treadmill), 6a (6M compare) · Model QP 5a (Fitbit) · CIE-2 Jan'25 4a · CIE-3 Jan'26 A4
2	Summarization techniques (sampling, distinct, frequency, membership)	SEE Feb'26 5b (6M), 6b (10M) · CIE-3 Jan'26 A5 & Q5 (10M) · Model QP 5b (reservoir) · CIE-2 Jan'25 1b (Bloom filter)
3	Caching strategies	Model QP 6a · CIE-2 Jan'25 5a (3 strategies, 10M)
4	Writing to long-term storage (3 ways) / batch loader	CIE-3 Jan'26 2b, 3a · Model QP 1.6
5	Stream time vs event time; trigger & eviction policy	Model QP 1.5 · CIE-2 Jan'25 A4, A5
6	Sliding-window support in frameworks	Model QP 6b · CIE-2 Jan'25 4a(ii)
7	In-memory database (IMDB/IMDG), constraints, concept drift	SEE Feb'26 1.6 · CIE-3 Jan'26 A3

1 · Querying a stream and accepting constraints **BOOK p.77** ↗

- **Ad-hoc query**: asked once about the stream (max value so far) — like an RDBMS query.
Continuous query: asked at all times (max value every 5 minutes, alert if above threshold).
BOOK p.77 ↗
- No common SQL-like language: Storm SQL (experimental), Samza none, Flink Table API (SQL-like), Spark SparkSQL/Hive. **BOOK p.78** ↗

Constraints you must accept [\(BOOK p.78 ↗\)](#)

Constraint	Meaning
One-pass	Data is not archived — you touch each item once ; multi-pass data-mining algorithms must be redesigned
Concept drift IMPORTANT CIE-3 Jan'26 A3	Statistical properties of the data change over time , hurting predictive models (e.g. a stock-price model trained pre-pandemic fails after it)
Resource constraints / load shedding	Arrival rate is not controllable; when overwhelmed the algorithm drops tuples (random or semantic load shedding)
Domain constraints	Business data itself is huge (100 M users' emails ⇒ petabytes) so even distinct counts are hard

Because of these, nearly every streaming method uses a **synopsis** (online summary) — accept **approximate** answers with high confidence. [\(BOOK p.79 ↗\)](#)

MNEMONIC — "One Cat Really Dislikes" = **One-pass · Concept drift · Resource limits · Domain constraints**

2 · Thinking about time [\(BOOK p.79 ↗\)](#)

2.1 Stream time vs event time [\(BOOK p.80 ↗\)](#) **IMPORTANT** Model QP 1.5 (2M) · CIE-2 Jan'25 A4 (2M, with example)

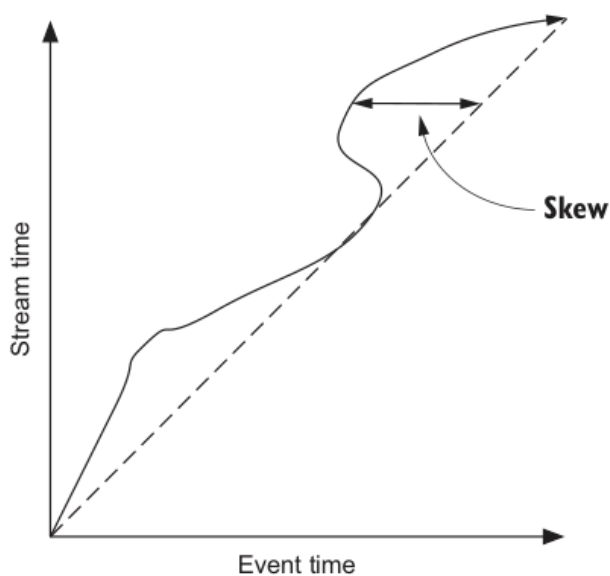


Figure 5.1 Time skew between event time and stream time

Fig 5.1 Time skew between event time and stream time (p.80)

Stream time	Event time
Time at which the event enters the streaming system (analysis tier)	Time at which the event actually occurs (e.g. on the device)
Fitbit: the reading reaches the analysis tier	Fitbit: the moment the runner's reading was recorded on the wrist

The gap is **time skew** (can be significant). Windows built on stream time give wrong averages if events arrive late.

⚠ **CONFLICT / TRAP — different "time" vocabularies**

Streaming Data book: **stream time vs event time** (use these two words in Unit 3 answers).
The Kafka book (Unit 5, Ch.14) uses **event time, log-append (ingestion) time, processing time**; its "processing time" $\approx$ the book's stream time, and it calls processing time "unreliable and best avoided". Don't mix them in one answer.

2.2 Windows of time (BOOK p.80 ↗)

- The engine cannot keep the infinite stream in memory, so computations run on **windows** — a small amount of data at a point in time (Fig 5.2). Every windowing technique has two policies: **Trigger policy** = rule that tells the system **when to run** your code on the window; **Eviction policy** = rule that decides **when data leaves** the window. Both are driven by **time or count**. (BOOK p.81 ↗) **IMPORTANT** CIE-2 Jan'25 A5

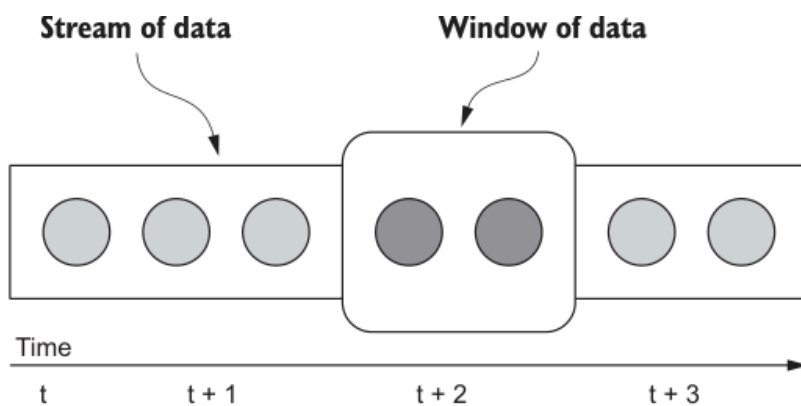


Figure 5.2 A window of data in perspective to the rest of the stream

Fig 5.2 A window of data in perspective to the rest of the stream (p.81)

2.3 Sliding window (BOOK p.81 ↗) **IMPORTANT** Model QP 5a (10M) · CIE-2 Jan'25 4a (10M) · SEE Feb'26 6a

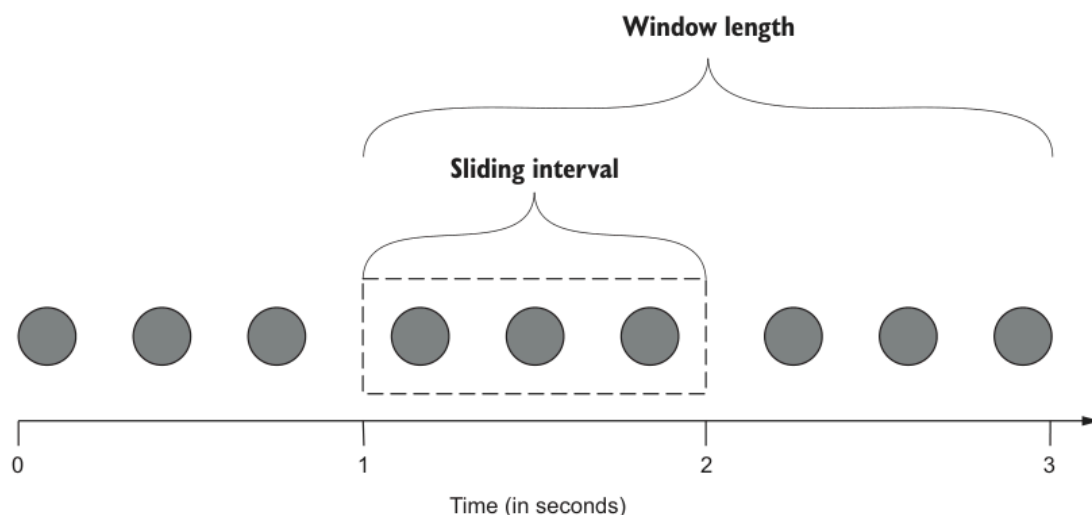
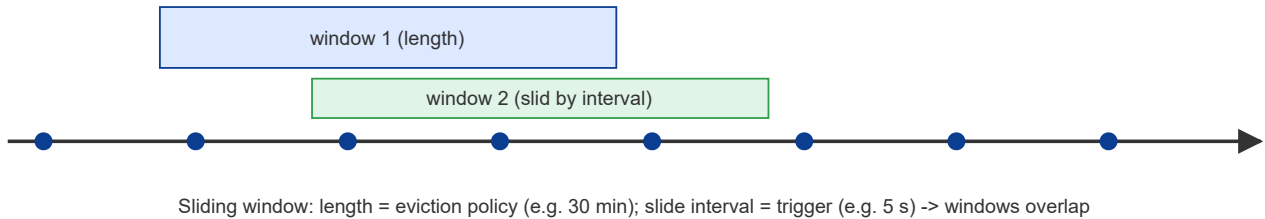


Figure 5.3 Sliding window showing the slide interval and the window length

Fig 5.3 Sliding window: window length = 2 s, sliding interval = 1 s (p.81)

- Policies are **time-based**: **window length = eviction policy** (how long data is kept; older data evicted) and **sliding interval = trigger policy** (how often your code runs). Windows **overlap** when interval $<$ length, so one event can appear in several windows.

- **Fitbit example (book p.82):** dashboard of **average speed per age group** (12–17, 18–24, 25–34...), updated **every 5 s**, averages over the **last 30 min** ⇒ **sliding window, length = 30 minutes, sliding interval = 5 seconds**. Because of time skew, window should use **event time** if the framework allows.



Sliding window diagram (our redraw): length vs slide interval, overlapping windows

Framework support — Table 5.2 [BOOK p.82](#) **IMPORTANT** Model QP 6b · CIE-2 Jan'25 4a(ii)

Framework	Sliding window	Event or stream time	Comment
Spark Streaming	Yes	Stream time only	No custom policies
Storm	No (native)	N/A	Can be implemented using timers
Flink	Yes	Both	User-defined custom trigger/eviction policies
Samza	No	N/A	No direct support

2.4 Tumbling window [BOOK p.83](#) **IMPORTANT** SEE Feb'26 5a (10M), 6a (6M) · CIE-3 Jan'26 A4 (2M)

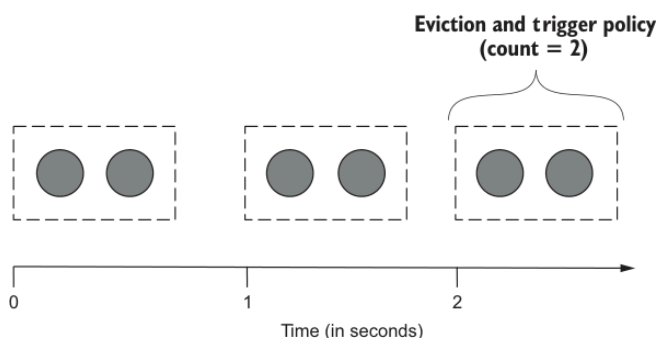


Figure 5.4 Count-based tumbling window with an eviction and trigger policy of two

Fig 5.4 Count-based tumbling, trigger = eviction = 2 items (p.83)

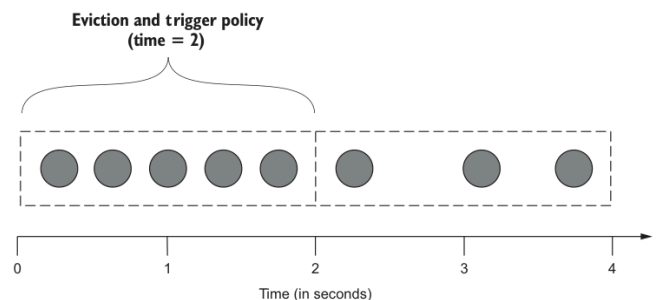


Figure 5.5 Temporal tumbling window with an eviction and trigger policy of two seconds

Fig 5.5 Temporal tumbling, trigger = eviction = 2 s (p.83)

- **Eviction policy = window full** (always). Trigger = count of items **or** time. When it fires, the window is **drained completely** ⇒ windows **never overlap**; each event is in exactly one window.
- Two types: **count-based** (fires when N items, irrespective of time) and **temporal-based** (fires when T time passes, irrespective of item count).

- **Book example (bicycles):** average speed every 30 s $\Rightarrow$ temporal tumbling window 30 s; alert every time > 100 people ride in a city $\Rightarrow$ count-based tumbling window of 100, one stream/window per city. To split by geography later keep the full message (GPS) rather than pre-filter. ([BOOK p.84 ↗](#))

Framework support — Table 5.3 ([BOOK p.84 ↗](#))

Framework	Count	Temporal	Comment
Spark Streaming	No	No	You must build it
Storm	Yes	Yes	No native windowing but easy to implement
Flink	Yes	Yes	Built-in support for both
Samza	No	Yes	(comment in book mistakenly says "sliding")

⚠ **CONFLICT / TRAP — Table 5.3 vs the text on page 85, and "sliding" vs "hopping"**

(a) Text on p.85 says Flink is **the only** framework with built-in tumbling support, while the table lists Storm Yes/Yes (implemented easily, not native) — write "Flink has built-in; Storm can implement easily".

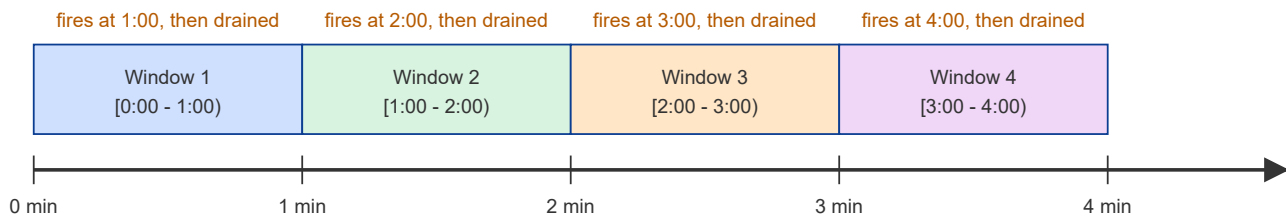
(b) The Kafka book (Unit 5) calls a fixed-size window that advances by less than its size a **hopping window** (tumbling = advance equals size) — e.g. its stock example uses a **5 s window advancing every 1 s**. The Streaming Data book (and the syllabus) call this a **sliding window**. In Unit 3 answers say "sliding"; in Kafka Streams answers say "hopping (what Unit 3 calls sliding)".

Sliding vs tumbling (**IMPORTANT** [SEE Feb'26 6a \(6M\)](#))

	Sliding	Tumbling
Eviction policy	Time-based window length (old data evicted as new arrives)	Window full (all items drained)
Trigger policy	Time-based sliding interval	Count or time
Overlap	Overlapping windows; event in many windows	Non-overlapping ; event in one window
Types	One (time-based)	Count-based, temporal
Output rate	Frequent, smooth moving results	One independent result per window
Use cases	Moving average over last 30 min refreshed every 5 s (Fitbit speed dashboard)	Total energy every 1 min; bikes' average speed every 30 s; alert when 100 riders
Framework support	Spark, Flink native	Flink native (Storm, Samza partial)

2.5 Worked answer — smart treadmills ([SEE Feb'26 Q5a, 10M](#)) (**IMPORTANT**)

Two metrics: (1) total energy consumption of all treadmills every 1 minute, one independent result per minute, no overlap, global for now (may split by gym later); (2) detect any treadmill exceeding a critical temperature in a 1-minute interval, reset every minute, non-overlapping.



Tumbling window: size = 1 min, slide = 1 min, no overlap; every reading belongs to exactly one window

Temporal tumbling window of 1 minute for the treadmill streams (our diagram)

1. **Window type: temporal tumbling window, size 1 minute** (trigger = eviction = 1 min).
Justification: results must be **independent, non-overlapping** and reset every minute $\Rightarrow$ eviction when the window is full/time elapsed; a sliding window would overlap and double-count energy.
2. **Metric 1:** create a stream carrying `power consumption` (keep the complete message incl. gym location); at each 1-minute trigger **sum** all tuples in the window $\Rightarrow$ total energy for that minute; drain window.
3. **Future split by gym:** because full messages are kept, group by gym id inside the window; avoid pre-filtering by location in the collection tier (inflexible, boundaries must be decided up front — book p.84).
4. **Metric 2:** on the same 1-min tumbling window, filter tuples with `machine temperature > threshold`; if any exist at the trigger, raise a maintenance alert; window resets so the next minute is clean.
5. **Time:** use event time (sensor timestamp) if the framework supports it to handle late tuples; otherwise stream time. **Framework:** Flink has built-in temporal tumbling windows (Table 5.3).

🔥 HOW TO WRITE IT — Fitbit question (Model QP 5a / CIE-2 Jan'25 4a)

Sliding window, **length 30 min, slide 5 s**, group by age band, average speed; draw Fig 5.3 style diagram labelled with those numbers; discuss stream vs event time (skew from device delay) and framework support (Flink: both times + custom policies; Spark: stream time only).

3 · Summarization techniques BOOK p.85 IMPORTANT SEE Feb'26 5b, 6b · CIE-3 Jan'26

A5, Q5 · Model QP 5b

Why summarise? The stream is unbounded and cannot fit in memory, so exact answers are costly or impossible; a good **estimate with high confidence** is usually enough (and the "picture of the business" doesn't change). Four techniques, each answers one question:

Technique	Question answered	Algorithm	Error type
Random sampling	Give me a random sample of the stream	Reservoir sampling	—
Counting distinct elements	How many <i>distinct</i> items?	Bit-pattern: LogLog, HyperLogLog , HLL++ · Order statistics: MinCount, Bar-Yossef	Approximate ($\approx 2\%$ with 1.5 KB for 1 billion)
Frequency	How many times has element X occurred?	Count-Min Sketch	Never under-counts, may over-count
Membership	Has X occurred before?	Bloom filter	False positives possible, no false negatives

🗨️ **MNEMONIC — "SD-FM" → Sampling, Distinct, Frequency, Membership = "Reservoir, HyperLogLog, Count-Min, Bloom"**

3.1 Random sampling — reservoir sampling [BOOK p.86 ↗](#) **IMPORTANT** Model QP 5b (6M)

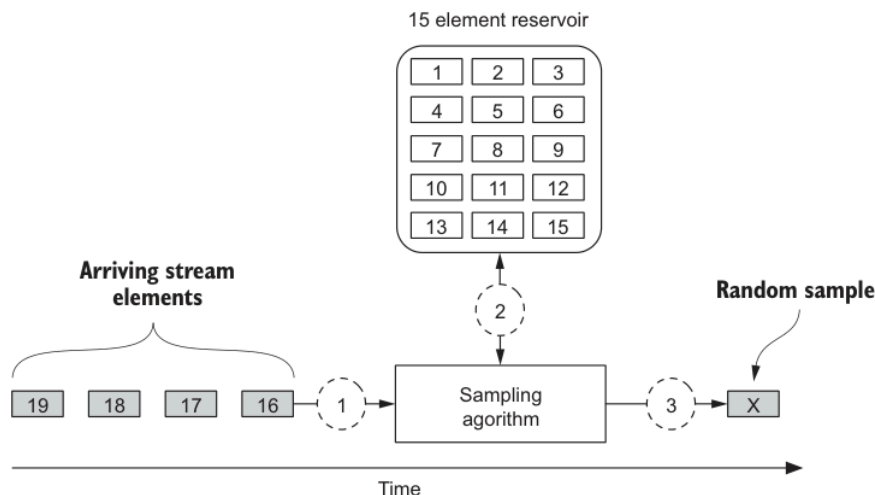


Figure 5.6 General flow of reservoir sampling with first new data item about to be processed

Fig 5.6 General flow of reservoir sampling; first new item (16th) about to be processed (p.86)

- Problem: ad network, 10 M ad views/min, statistical analysis needed, data doesn't fit memory or disk $\Rightarrow$ keep a **reservoir of fixed size k** .
- Fill the reservoir with the **first k** items. For the **n -th** item ($n > k$): include it with probability **k/n** ; if chosen, it **replaces a randomly chosen** element already in the reservoir. Every element seen so far has equal probability **k/n** of being in the reservoir.
- Book example ($k = 15$): the 16th item enters with probability **$15/16$** (generate a random number in $[0,1]$; if $< 15/16$ add it and displace a random reservoir item). [BOOK p.87 ↗](#)
- Not built into Spark/Storm/Samza/Flink but easy to implement. Reference: Vitter 1985.

3.2 Counting distinct elements — HyperLogLog [BOOK p.87 ↗](#)

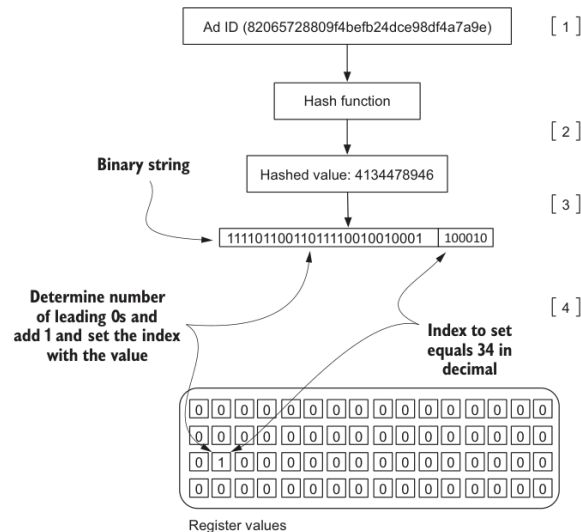


Figure 5.7 Processing a single stream element with the HyperLogLog algorithm

Fig 5.7 Processing one stream element with HyperLogLog (p.89)

1. Take the element (ad ID) → **hash** it to a binary string.
 2. The **last 6 bits** (precision = 6) give the **register index** (100010 = 34).
 3. Count the **leading zeros** of the remaining bits + 1; store it at that register if larger.
 4. Estimate = **harmonic mean** of all register values (averaging many estimates reduces bias; harmonic mean reduces outlier impact).
- Uses tiny space (**1 billion distinct items at 2% accuracy in 1.5 KB**) and is **distributable**: union of two HLL structures ⇒ merge per-node summaries. HLL++ = less memory, better accuracy. Needed e.g. for a 40 Gbps intrusion-detection system (~78 M packets/s).

[BOOK p.90 ↗](#)

3.3 Frequency — Count-Min Sketch [BOOK p.90 ↗](#)

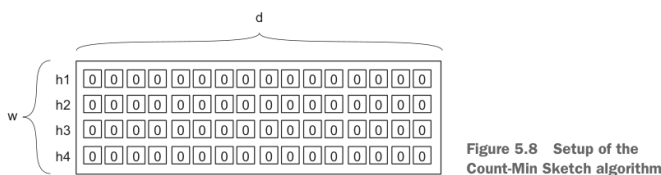


Fig 5.8 Setup: $w \times d$ counters, one hash per row (p.91)

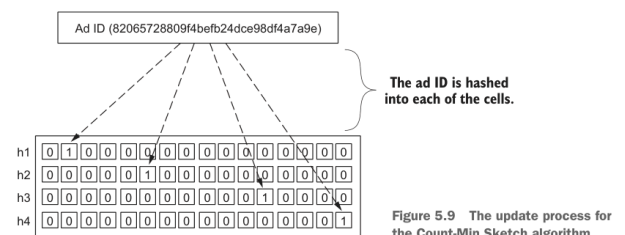


Fig 5.9 Update: hash into each row, add 1 (p.91)

- 2-D array of counters initialised to 0; each **row has its own pairwise-independent hash function** (example 4×16).
- **Update**: hash the element with each row's hash, **increment** that cell in every row. **Query**: **estimate = min** of the cells the element hashes to — the minimum reduces collision error. **Never under-counts, may over-count.**
- Answers **point queries** (one element), **range queries**, **inner-product queries** (join size of two sketches). Accuracy: 8×128 array ⇒ $\approx 1.5\%$ relative error with 99.6% probability. Uses: how often an ad was viewed; DoS detection. [BOOK p.92 ↗](#)

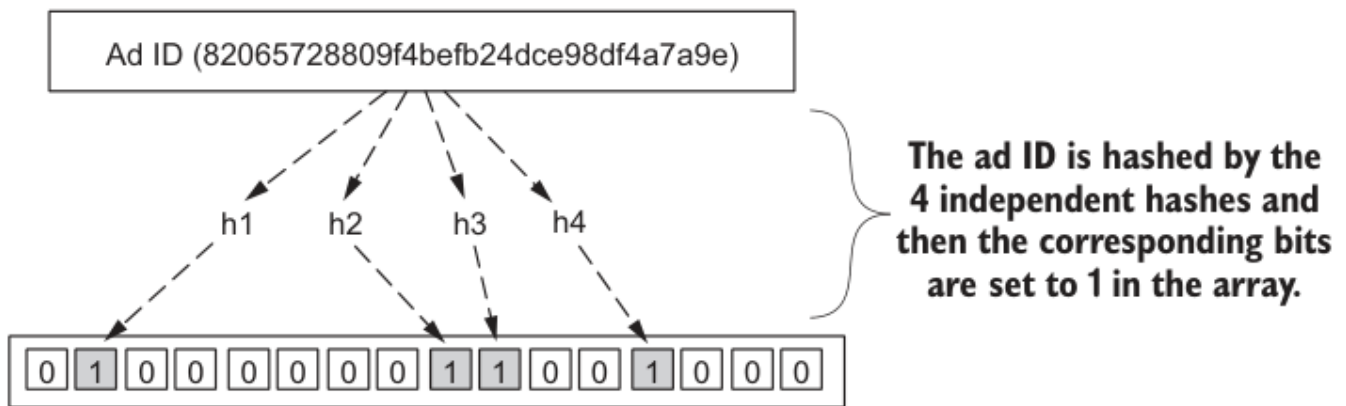


Figure 5.10 A Bloom filter showing one stream element being processed

Fig 5.10 A Bloom filter processing one element: 4 hashes set 4 bits to 1 (p.93)

- Binary **bit array of length m** (space = $m/8$ bytes) + several independent hash functions.
Insert: hash the element with each function and **set those bits to 1** (a set bit stays set on collision).
- **Query:** compute the hashes; if **all** bits are 1 $\Rightarrow$ "probably seen", if **any** is 0 $\Rightarrow$ **definitely never seen**. So **false positives possible, false negatives impossible**.
- **Membership queries** ("does this element belong to the set/group?") are exactly what it solves: check if an ad/packet signature/bot host has appeared **before paying for an expensive lookup** in an external store; e.g. network monitoring first checks the filter for known bad hosts, then confirms positives with the costly check. [BOOK p.94 ↗](#)

Compare the structures

	Count-Min Sketch	Bloom filter
Structure	Several counter arrays (width w, length m), 1 hash per array	One bit array length m, several hashes
Update	Increment counters	Set bits to 1
Answer	Approx. count (min of counters)	Yes/no membership (AND of bits)
Error	Over-estimates only	False positives only

HOW TO WRITE IT — "Role of summarisation with example and classification" (SEE 5b, 6M) and "list and elaborate any two" (SEE 6b, 10M)

5b: why (unbounded stream, memory, approximate OK) $\rightarrow$ table of 4 classes with the question each answers + one example (ad network: sample views, distinct ads/min, views of ad X, was ad X shown before?). 6b: list 4, then elaborate two (choose **reservoir sampling** and **Count-Min/Bloom** — steps are easy to draw; give formula k/n or min-of-counters) with a diagram from Fig 5.6/5.9/5.10 and one example each, 5 marks each.

4 · Storing the analysed or collected data BOOK p.95 ↗

After analysis there are **four options**: (1) **analyse and discard**; (2) **analyse and push back into the streaming platform** (output of one job = input of another; core of Samza); (3) **analyse and store for real-time usage**; (4) **analyse and store for batch/offline access**. BOOK p.95 ↗

IMPORTANT CIE-3 Jan'26 3a

4.1 Long-term storage — three ways to write BOOK p.97 ↗ **IMPORTANT** CIE-3 Jan'26 2b (6M), 3a (6M)

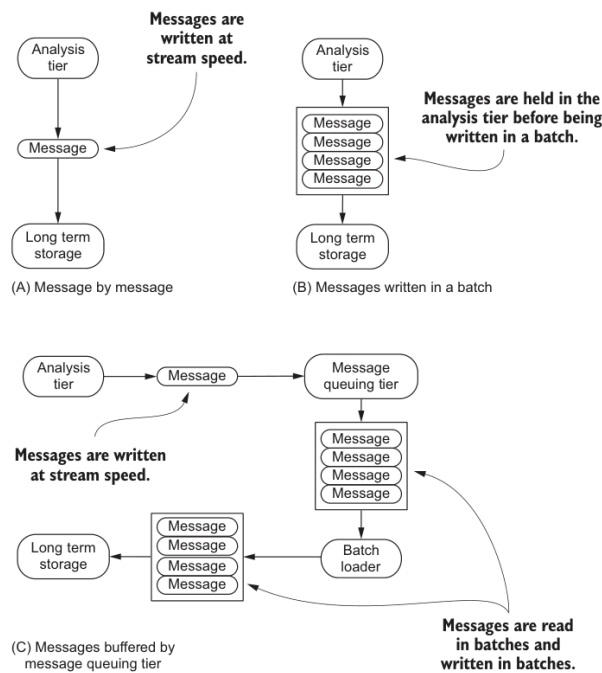


Figure 6.2 Three ways to write data to long-term storage

Fig 6.2 Three ways to write data to long-term storage (p.97) — DRAW THIS

Option	How	Type	Risk / benefit
A. Message by message	Write each message as processed — at stream speed	Direct writing	Store may not keep up ⇒ hurts stream processing; batch stores (HDFS, S3) are not made for continuous writes
B. Batch from analysis tier	Hold messages in analysis tier until batch size or time reached, then write	Direct writing	Optimises for slow writes but still couples analysis to store speed
C. Via message queuing tier + batch loader	Analysis tier writes to queue at stream speed; batch loader reads in batches and writes in batches	Indirect writing	Decouples analysis from store performance; queue is built for stream speed; loader is dedicated bulk-load tool; extra component = complexity

- **Batch loader** = component that reads batches of messages from the queue tier and writes them to long-term storage; typical ETL features: scheduling, error handling, data-quality checks, publishing, monitoring, scaling, fault tolerance. Tools: **Gobblin** (LinkedIn), **Secor** (Pinterest) for Kafka → HDFS/S3. BOOK p.98 ↗ **IMPORTANT** Model QP 1.6 (define, one benefit)
- **Video-analytics (CIE-3 3a):** CCTV results for compliance and model training ⇒ option C is best (queue absorbs frame-result bursts, loader writes Parquet/files to S3/HDFS); A or B for

low volume.

4.2 Keeping it in memory [BOOK p.99 ↗](#)

- Goal: act on data in real time, so current/recent data must be in memory ("RAM is the new disk"). SSD seek $\approx 100,000$ ns vs DRAM ≈ 100 ns vs L1 ≈ 0.5 ns; a 1 TB DRAM cluster $\approx$ \$5,000/month on EC2. [BOOK p.100 ↗](#)
- Three categories: **embedded in-memory / flash-optimised**, **caching systems**, **IMDB / IMDG**.

(a) Embedded in-memory / flash-optimised [BOOK p.100 ↗](#)

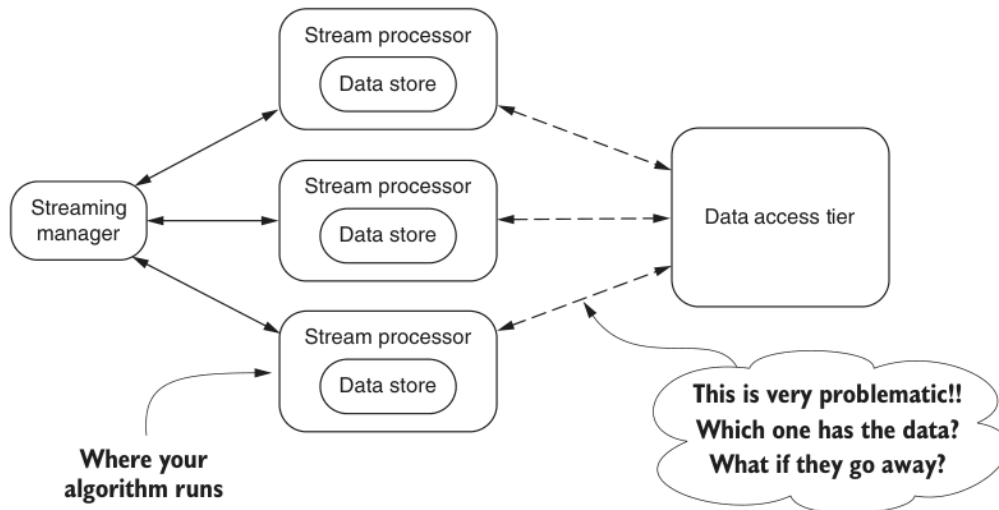


Figure 6.3 Architectural view of using an embedded data store with our analysis nodes

Fig 6.3 Embedded store on each analysis node — "very problematic" (p.100)

- Single-node, embedded in your software, no cross-node access/management $\Rightarrow$ **poor fit** for distributed streaming: which node has the data? what if it goes away? Nodes then do processing + storage + serving (fragile). Products: SQLite, RocksDB, LevelDB, LMDB, Perst.

(b) Caching systems — strategies [BOOK p.101 ↗](#) **IMPORTANT** Model QP 6a · CIE-2 Jan'25 5a (any three, 10M)

Object cache / in-memory key-value store, often **DRAM only**. Strategies related to **persisting cache entries and keeping them fresh**:

Strategy	How it works	Pros / cons
Read-through (Fig 6.4)	On a miss the cache loads from the persistent store transparently to the client	Penalty on first read; updates must still be written to store
Refresh-ahead (Fig 6.5)	Cache refreshes recently used entries before they expire	Avoids read-through penalty, fresher; hard to sync with a fast stream
Write-through (Fig 6.6)	Cache writes through to the backing store ; write acknowledged only after the store write	No separate sync process; extra write latency
Write-around (Fig 6.7)	Store updated out-of-band ; another process updates the cache afterwards	Cache needn't talk to store; dual update complexity
Write-back / write-behind (Fig 6.8)	Write acknowledged after cache; store updated in the background	No I/O wait; risk of data loss until flushed

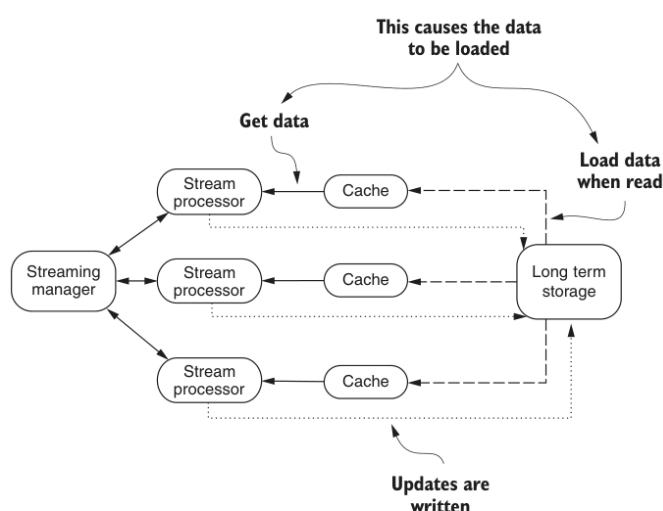


Figure 6.4 Read-through caching strategy showing how the data is loaded from long-term storage if it's not in the cache

Fig 6.4 Read-through (p.102)

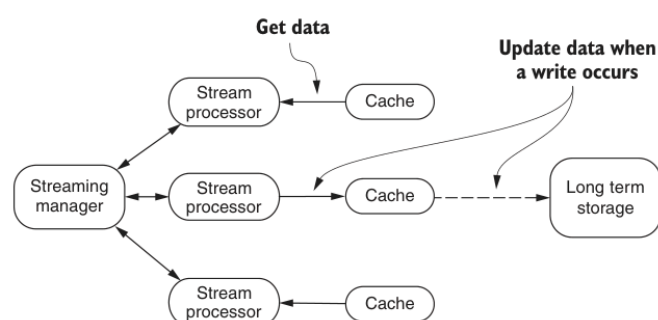


Figure 6.6 Write-through cache strategy showing how data is written all the way through to the long-term storage

Fig 6.6 Write-through (p.103)

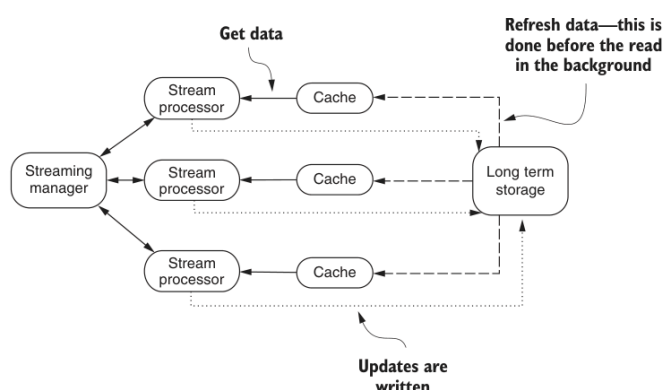


Figure 6.5 Refresh-ahead cache strategy, showing how data is refreshed from long-term memory

Fig 6.5 Refresh-ahead (p.103)

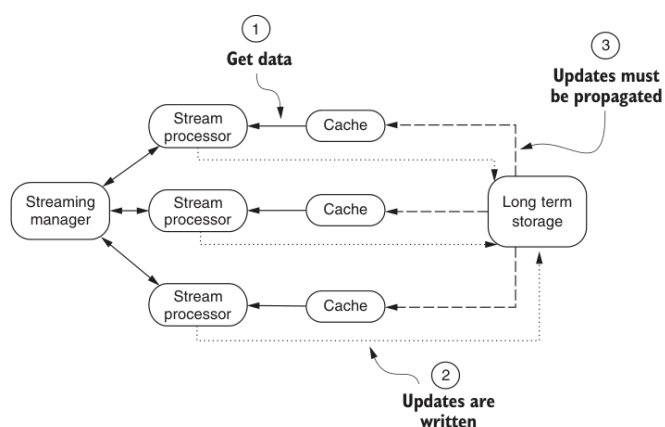


Figure 6.7 Write-around caching strategy showing how data is written to long-term storage and then has to be propagated to the cache

Fig 6.7 Write-around (p.104)

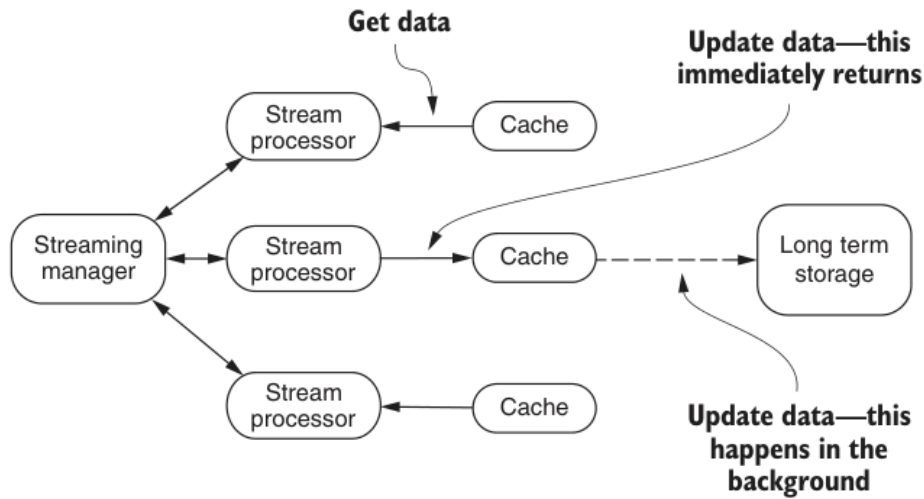


Figure 6.8 Write-back caching strategy

Fig 6.8 Write-back / write-behind (p.104)

- Risks: data only in memory $\Rightarrow$ losing a cache node loses data; dual writing of a constant stream is a **bottleneck**. Products: **Memcached** (use write-around), **EHCache** (read/write-through, write-behind), **Hazelcast** (read/write-through), **Redis** (own persistence; you build the strategy). ([BOOK p.105 ↗](#))

🧠 **MNEMONIC — "RRW-WWW": Read-through, Refresh-ahead | Write-through, Write-around, Write-back**

(c) In-memory database (IMDB) and in-memory data grid (IMDG) ([BOOK p.105 ↗](#)) **IMPORTANT**

SEE Feb'26 1.6

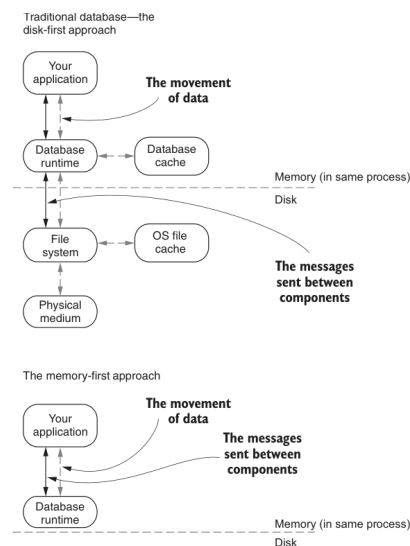


Figure 6.9 The two different approaches to database design: disk-first and memory-first

Fig 6.9 Disk-first (traditional DB) vs memory-first design (p.106)

- **Purpose of an IMDB:** keep the working data **memory-first** for very fast real-time reads/writes while still using **disk for durability** (logging + periodic snapshots) and replication — so analysis results can be served in real time to clients (e.g. session personalisation) without losing data on a crash.
- Unlike caches, they persist to disk; unlike traditional DBs (SQL Server, Oracle, Cassandra — **disk-first, memory-second**) they are **memory-first, disk second**. IMDG = distributed,

compute close to data, non-SQL API; IMDB = often SQL API; the lines are blurring.

- Products: MemSQL, VoltDB, Aerospike (TTL per record $\Rightarrow$ session expiry), Apache Geode, Couchbase, Apache Ignite, Hazelcast, Infinispan. [BOOK p.107 ↗](#)

Category	Persistence	Distributed?	Fit for streaming
Embedded	Local	No	Poor
Caching system	Usually none (DRAM)	Often	OK for transient results; dual-write cost
IMDB / IMDG	Disk logging + snapshots, replicated	Yes	Best (in-session personalisation use case)

Use case (§6.3, p.108): in-session personalisation (30-minute session, click-stream: show helmets matching yellow sunglasses; coupon for sunglasses removed from cart) $\Rightarrow$ embedded = bad, cache = thrashing (read-update-write whole session), **IMDB/IMDG = good**. Energy smart-grid exercise p.111.

5 · Quick revision — Unit 3

Ad-hoc vs continuous query?	Once vs always-on.
Four constraints?	One-pass, concept drift, resource constraints (load shedding), domain constraints.
Stream time vs event time?	Enters system vs occurs at source; gap = time skew.
Trigger vs eviction policy?	When to process vs when to drop data from window.
Sliding window controlled by?	Window length (eviction) + sliding interval (trigger).
Tumbling window types?	Count-based, temporal.
Fitbit dashboard window?	Sliding, length 30 min, interval 5 s.
Which framework supports sliding + event time?	Flink (Spark = stream time only).
Reservoir sampling probability for n-th item?	k/n .
HyperLogLog counts what?	Distinct elements (leading zeros of hash; harmonic mean).
Count-Min Sketch error?	Never under-counts; estimate = min over rows.
Bloom filter error?	False positives yes, false negatives no.
Three ways to write to long-term store?	Message-by-message, batch from analysis tier, via queue + batch loader.
Batch loader tools?	Gobblin, Secor.
Cache strategies?	Read-through, refresh-ahead, write-through, write-around, write-back.
Disk-first vs memory-first?	Traditional DB vs IMDB/IMDG.
Which cache needs write-around?	Memcached.